

AirOne V7.1 — Connecting a Real CanSat (LoRa E22 over USB)

This guide explains how to receive **live telemetry** from a physical CanSat on your laptop using a **LoRa E22** module connected via a **USB-to-serial** bridge. It covers wiring, E22 configuration, launching the ground station in live mode, and troubleshooting.

Scientific-honesty guarantee. The ground station never fabricates telemetry. If no radio is found it reports an explicit `NOT_CONFIGURED` state and streams nothing. Simulation (`--simulate`) is an isolated testing/training mode only; real telemetry is always authoritative and simulated packets are tagged `SIMULATED` .

1. Signal chain overview

```
[CanSat: ESP32 + sensors + E22 (air)]  ~~LoRa RF~~  [E22 (ground) + USB-serial]  ->
laptop USB
```

|

(launcher.py)

AirOne ground station

- The **flight E22** is driven by the ESP32 firmware in `firmware/airone_cansat/` .
- The **ground E22** is wired to a USB-serial bridge (CP210x / CH340 / FTDI) and appears on your laptop as a serial port.
- Both E22 modules must share the **same** RF settings (frequency channel, address, air data rate) and the **same UART baud** as the ground station.

2. Ground-side wiring (E22 ↔ USB-serial bridge)

Put the E22 in **transparent mode** (M0 = M1 = GND). In transparent mode the E22 passes bytes through unchanged — exactly what the AirOne binary protocol needs.

E22 pin	Connect to (USB-serial bridge)
VCC	3.3 V (E22 is 3.3 V logic)
GND	GND
TXD	RXD of the bridge
RXD	TXD of the bridge
M0	GND (transparent mode)
M1	GND (transparent mode)
AUX	(optional) leave unconnected for basic RX

Use a 3.3 V USB-serial bridge. Feeding 5 V into E22 RXD can damage the module.

3. Flight-side wiring (ESP32 ↔ E22)

Defaults used by `firmware/airone_cansat/airone_cansat.ino` :

ESP32 pin	E22 pin	Notes
GPIO17 (TX2)	RXD	ESP32 → E22
GPIO16 (RX2)	TXD	E22 → ESP32
3.3 V	VCC	
GND	GND, M0, M1	transparent mode

Sensors (I²C on SDA=21, SCL=22) and the MAX-M10S GNSS (UART1: GPIO4/GPIO2) are detailed in `firmware/README.md`.

4. E22 baud rate must match

The E22's **UART baud** (module setting) and the AirOne **ground-station baud** must be identical. The firmware and ground station both default to **115200**. If your E22 modules are configured for a different UART baud (common factory default is 9600), either reconfigure them or pass the matching `--baud`.

Because the E22 in transparent mode has a **fixed** UART baud, use `--force-baud` so the ground station opens the port directly at that baud instead of trying to auto-negotiate:

```
python launcher.py --serial-port /dev/ttyUSB0 --baud 115200 --force-baud
```

5. Finding the serial port

- **Linux:** usually `/dev/ttyUSB0` (FTDI/CP210x) or `/dev/ttyACM0`.
List devices: `ls /dev/ttyUSB* /dev/ttyACM*` or `dmesg | grep tty` after plugging in.
You may need permission: `sudo usermod -aG dialout $USER` then re-login.
- **macOS:** `/dev/tty.usbserial-XXXX` or `/dev/tty.SLAB_USBtoUART`.
- **Windows:** COM3, COM5, ... (see Device Manager → Ports).

Auto-discovery: if you omit `--serial-port`, AirOne scans for known USB-serial bridges (CP210x 0x10C4, CH340 0x1A86, FTDI 0x0403) and picks the first match.

6. Launching the ground station in live mode

Set the mandatory JWT secret (≥32 chars) and run the launcher:

```
export AIRONE_JWT_SECRET="your-long-random-secret-at-least-32-characters"

# Auto-discover the radio:
python launcher.py

# Or specify the port/baud explicitly (recommended for E22 transparent mode):
python launcher.py --serial-port /dev/ttyUSB0 --baud 115200 --force-baud
```

On startup the launcher prints the explicit link state, one of:

- **LIVE** (`/dev/ttyUSB0 @ 115200 baud`) — receiving live telemetry.
- **NOT_CONFIGURED** (no CanSat radio found ...) — nothing connected; **no data is fabricated**. Plug in the radio (or use `--simulate` for isolated testing).
- **DISABLED** (`telemetry.enabled=false`) — link intentionally turned off.

The live link status is also available in the health report / API and updates `bytes_received`, `packets_received`, `crc_errors`, and `reconnect_count`.

Configuration reference (`config/default_config.yaml`)

```
telemetry:
  enabled: true           # false = do not start the live link
  serial_port: null       # null = auto-discover; or "/dev/ttyUSB0" / "COM5"
  baud_rate: 115200      # must match the E22 UART baud
  force_baud: false       # true = open directly at baud_rate (E22 transparent)
```

CLI flags override config. `AIRONE_SERIAL_PORT` env var overrides the config `serial_port` (but is overridden by `--serial-port`).

7. What good telemetry looks like

Each frame the CanSat sends is CRC-protected. When frames arrive:

- `packets_received` climbs and `crc_errors` stays low.
- Valid frames flow through the 20-stage pipeline and appear in the GUI/API.
- Corrupted frames are counted as CRC errors and **discarded** (never accepted with fabricated values).

8. Troubleshooting

Symptom	Likely cause / fix
<code>NOT_CONFIGURED</code> but radio is plugged in	Wrong/aliased port — pass <code>--serial-port</code> explicitly; check <code>ls /dev/ttyUSB*</code> . On Linux add yourself to <code>dialout</code> .
Port opens but <code>packets_received</code> stays 0	Baud mismatch — use <code>--baud</code> to match the E22 UART; add <code>--force-baud</code> . Confirm the CanSat is powered and transmitting.
Many <code>crc_errors</code>	RF interference / weak link / partial frames — check antennas, reduce distance, confirm both E22s share channel/address/air-rate.
Data arrives then stops	Link dropped — the receiver auto-reconnects with backoff; check power/USB. <code>reconnect_count</code> will increase.
<code>pyserial</code> not installed	<code>pip install -r requirements_v71.txt</code> (installs <code>pyserial</code>).
Bytes look garbled in a serial monitor	That's the binary frame format — expected. Do not use a text serial monitor to validate; use the AirOne ground station.
Permission denied on <code>/dev/ttyUSB0</code> (Linux)	<code>sudo usermod -aG dialout \$USER</code> , then log out/in.

9. Testing without hardware (isolated simulation)

When you have no radio at hand, exercise the entire real pipeline with the physics-based simulator. Simulated packets use the identical binary framing and are tagged `SIMULATED`:

```
python launcher.py --simulate --sim-loop
```

If a live link is also up, real telemetry remains authoritative; the launcher warns that simulation is running alongside it.